

RAPPORT D'AUDIT

DE QUALITE ET DE PERFORMANCE

APPLICATION TODO & CO

1) Contexte des audits

L'application **Todo & Co** est une plateforme conçue pour aider les utilisateurs à organiser leurs tâches de manière simple et efficace. Au départ, elle a été développée sous une version de base (MVP : Produit Minimum Viable) pour tester les fonctionnalités essentielles. Cependant, au lieu de simplement améliorer cette version initial, l'application a été complètement recréé à partir de zéro en utilisant une version récente de l'outil de développement Symfony. Cette refonte a permis de construire une base solide pour rendre l'application plus performante et plus facile à maintenir.

A la demande des fondateurs, des audits ont été réalisé pour analyser à la fois la qualité du code et les performances de l'application avant et après ces améliorations.

L'**audit de qualité du code** s'est concentré sur des aspects clés tels que le respect des conventions de codage, la gestion des dépendances et l'optimisation des structures de classes et de méthodes.

En parallèle, un **audit de performance** a été mené pour comparer les données pour chaque page, comme le temps de réponse ou l'utilisation de la mémoire, afin de mesurer les progrès réalisés. L'objectif est de s'assurer que l'application fonctionne de manière fluide, rapide et réactive tout en soutenant une charge accrue des données.

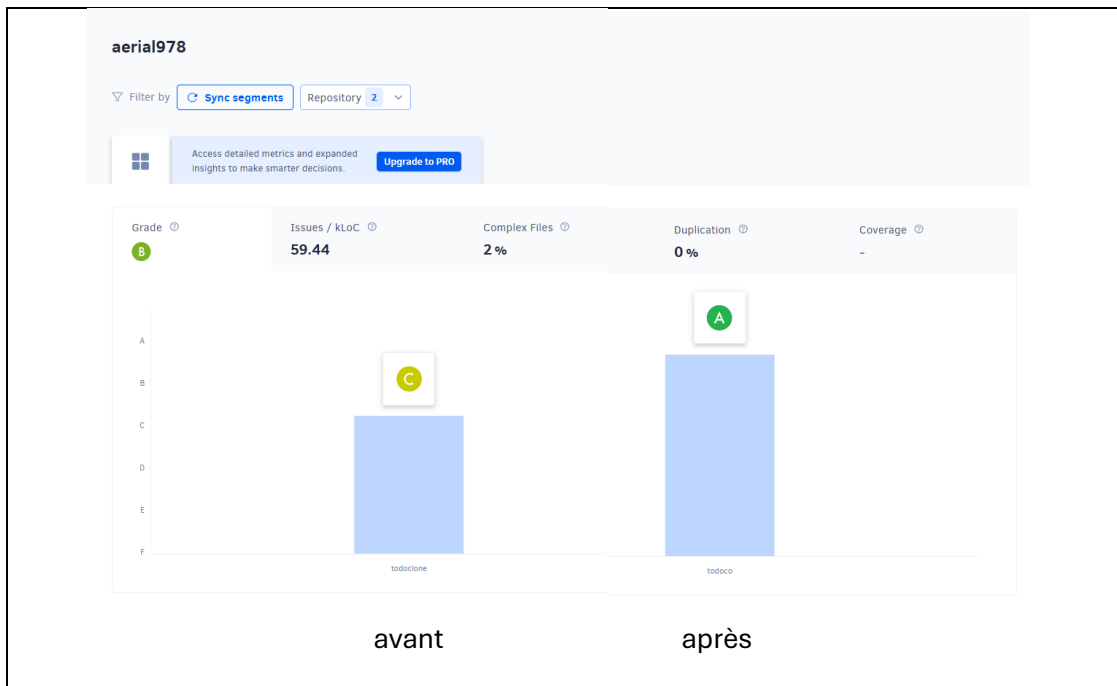
2) Audit de qualité du code

2.1) Choix de l'outil : Codacy

Il permet d'identifier automatiquement les problèmes liés à la maintenabilité, la duplication et la sécurité. Son intégration avec Github facilite le suivi des corrections et la comparaison des versions avant et après amélioration. Codacy fournit ainsi des indicateurs précis pour quantifier la dette technique et garantir un code plus propre et maintenable.

2.2) Analyse de la qualité du code

2.2.1) Grade (qualité du code)

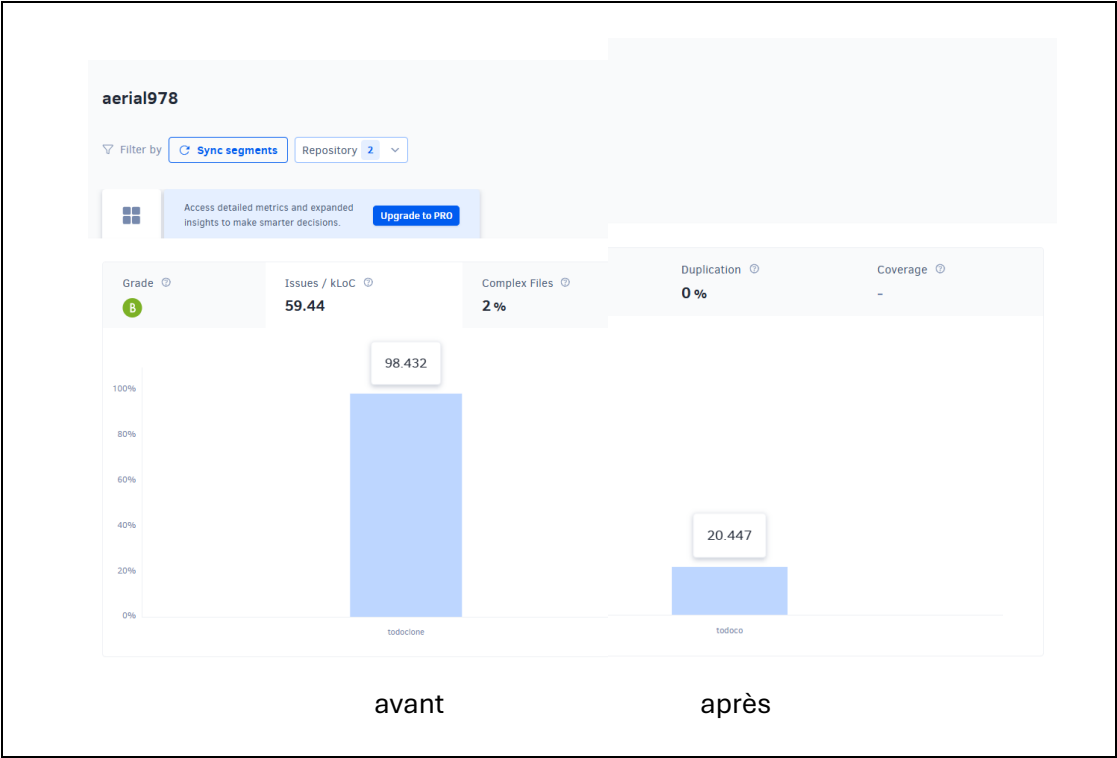


L'onglet « **Grade** » dans le **dashboard** de **Codacy**, représente un indicateur visuel rapide permettant de voir l'évolution de la qualité du code du projet. Il attribue une note allant de A (excellente qualité) à F (mauvaise qualité), en fonction des critères d'analyses suivantes :

- **Complexité du code** : plus le code est complexe, plus il est difficile à maintenir ;
- **Duplication de code** : si de nombreuses parties du code sont dupliquées, la maintenabilité est réduite ;
- **Erreurs de style et conventions** : Codacy vérifie si le code respecte les bonnes pratiques (indentation, espaces, commentaires ...) ;
- **Problème de sécurité** : présences de failles potentielles, comme l'utilisation de fonctions non sécurisées.

Avant amélioration, Codacy a attribué un **grade C** indiquant un code avec plusieurs problème de qualité et nécessitant des corrections. Le **grade A** montre une nette amélioration avec un code beaucoup plus propre, optimisé et conforme aux bonnes pratiques.

2.2.2) Erreurs pour 1000 lignes de code (KLoC : Kilo Lines Of Code)



Cet onglet « **issues** » représente le nombre total de problèmes en fonction de la taille du code. Au lieu de donner un nombre total d’erreurs, Codacy affiche un ratio. Cela permet de comparer la qualité du code indépendamment de la taille du projet.

Avant amélioration, il y avait environ **98 erreurs pour chaque 1000 lignes de code**. Après refactorisation, ce ratio est considérablement réduit à **20 erreurs**. Il est constaté **une diminution de 79%** du nombre d’issues. Ce graphique est une preuve chiffrée des bénéfices du refactoring, montrant que les corrections et les optimisations effectuées ont eu un effet majeur sur la qualité du projet.

2.2.3) Erreurs par gravité et par catégorie

	Avant amélioration	Après amélioration
Par gravité	Severity ^ Category ☐ MINOR 248 ☐ CRITICAL 24 ☐ MEDIUM 23 Clear Apply	Severity ^ Category ☐ MINOR 360 ☐ CRITICAL 4 ☐ MEDIUM 3 Clear Apply
Par catégorie	Category ^ Author ☐ Code style 253 ☐ Security 39 ☐ Unused code 3 Clear Apply	Category ^ Author ☐ Code style 360 ☐ Error prone 4 ☐ Security 3 Clear Apply

Par gravité, les erreurs **critiques** ont drastiquement diminué (-83%), ce qui indique une nette amélioration de la sécurité et de la robustesse du code. Les erreurs **mineures** ont augmenté (+112), ce qui peut s'expliquer par une amélioration des standards de qualité du code et l'ajout de nouvelles règles d'analyse.

Par catégorie, les problèmes de **sécurité** ont été presque totalement résolus (-91%). Les problèmes de **complexité** et de **code inutilisé** ont été supprimés. Les problèmes de **documentation et de style** ont augmenté, probablement en raison d'un renforcement des bonnes pratiques. Les erreurs de **mise en forme** ont légèrement augmenté, ce qui peut être corrigé facilement avec des outils d'autoformatage.

2.3) Conclusion de la qualité du code

L'audit de qualité du code montre une nette amélioration après refonte. Le grade Codacy est passé de C à A, traduisant un code plus propre et maintenable. Le nombre d'erreurs par kLoC a chuté de 98.432 à 20.447, réduisant la complexité et les failles potentielles. Les erreurs critiques et moyennes ont été largement corrigées, rendant l'application plus fiable. L'augmentation des erreurs mineures reflète un meilleur respect des conventions de codage. Le code est désormais plus clair, sécurisé et conforme aux standards, assurant une meilleure évolutivité.

3) Audit des performances

3.1) Choix de l'outil : Symfony Profiler

Pour cet audit, l'outil Symfony Profiler a été retenu comme solution d'analyse des performances. Intégré nativement dans le framework Symfony, le Profiler offre des capacité complète pour surveiller et évaluer les performances des applications web développées avec ce framework.

Le choix de cet outil repose sur plusieurs facteurs :

- Approche détaillée : il permet d'accéder à vue détaillé des différentes étapes du traitement d'une requête HTTP.
- Métriques clés : il offre des informations précises sur le temps d'exécution total, la consommation mémoire, et le traitement des événements et des requêtes SQL.
- Facilité de comparaison : les données collectées avant et après les améliorations peuvent être facilement comparées pour évaluer l'impact des modifications apportées.

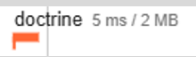
4) Analyse des performances des pages

4.1) Liste des tâches

	Avant amélioration			Après amélioration		
Performances métriques	204 ms	32 ms	2.00 MB	Total execution time	Symfony initialization	Peak memory usage
	Total execution time	Symfony initialization	Peak memory usage	150 ms	22 ms	4.00 MiB

L'analyse des métriques montre une nette amélioration des performances globales après la mise à jour. Le temps nécessaire pour charger la page et démarrer Symfony a été significativement réduit. Même si l'application utilise un peu de mémoire qu'avant, cela explique par l'ajout de nouvelles fonctionnalités utiles.

Plusieurs lignes clés de l'exécution timeline méritent d'être mises en avant :

	Avant amélioration	Après amélioration
doctrine		
templates		

Doctrine illustre une meilleure optimisation des interactions avec la base de données, réduisant la durée des opérations.

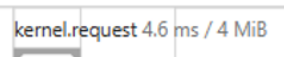
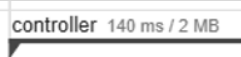
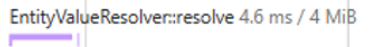
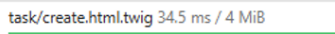
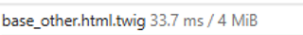
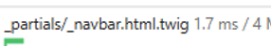
L'ajout d'une fonctionnalité importante comme la **pagination**, justifie une partie de l'augmentation de la complexité.

4.2) Création d'une tâche

	Avant amélioration	Après amélioration
Performances metrics	345 ms 86 ms 2.00 MB Total execution time Symfony initialization Peak memory usage	Total execution time Symfony initialization Peak memory usage 130 ms 24 ms 4.00 MiB

Après optimisation, le temps d'exécution total pour la page a été réduit de 345 ms à 130 ms, soit une amélioration de 62 %. L'initialisation de Symfony, initialement à 86 ms, a également été optimisée pour atteindre 24 ms. Bien que l'utilisation de mémoire ait doublé, passant de 2 MB à 4 MB, cela est justifié par l'enrichissement fonctionnel de l'application. Globalement, les améliorations ont rendu l'application plus rapide et plus performante, avec une augmentation maîtrisée de la consommation des ressources pour offrir de nouvelles fonctionnalités.

voici les lignes clés à mettre en avant dans l'exécution timeline :

	Avant amélioration	Après amélioration
kernel.request		
controller		
EntityValueResolver		
templates	 	  

Réduction drastique de **kernel.request**, montrant l'amélioration de l'initialisation et de la gestion des middlewares.

Optimisation du **controller**, réduction des surcharges grâce à un refactoring de la logique métier.

Ajout de la fonctionnalité **EntityValueResolver** démontrant une modernisation de l'approche technique.

Augmentation du temps des **templates** indiquant des efforts pour enrichir l'expérience utilisateur, en contrepartie d'une charge plus importante, mais acceptable.

4.3) Modification d'une tâche

	Avant amélioration			Après amélioration		
Performances metrics	321 ms Total execution time	30 ms Symfony initialization	4.00 MB Peak memory usage	Total execution time 189 ms	Symfony initialization 24 ms	Peak memory usage 4.00 MiB

Baisse de 41 % du temps d'exécution, la mémoire reste stable, reflétant une gestion plus rapide des requêtes sans augmentation significative de la consommation de ressources.

Lignes à mettre en avant dans la timeline :

	Avant amélioration	Après amélioration
kernel.request	kernel.request 73 ms / 2 MB	kernel.request 5.3 ms / 4 MiB
controller	controller 160 ms / 2 MB	controller 79.4 ms / 4 MiB
ControllerListener & ParamConverterListener	Sensio\Bundle\FrameworkExtraBundle\Event\Listener\ControllerListener 17 ms / 2 MB Sensio\Bundle\FrameworkExtraBundle\Event\Listener\ParamConverterListener 16 ms / 2 MB	
templates	task/edit.html.twig 7 ms / 2 MB base.html.twig 7 ms / 2 MB	task/edit.html.twig 29.3 ms / 4 MiB base_other.html.twig 28.2 ms / 4 MiB _partials/_navbar.html.twig 2 ms / 4 MiB

kernel.request indique une optimisation spectaculaire des processus initiaux, comme le routage pour réduire le délai.

La réduction du temps de traitement dans le **contrôleur**, met en évidence une externalisation réussie des logiques métier, auparavant lourdes et non découpées, vers des services ou entités dédiées.

ControllerListener & **ParamConverterListener** indiquent des traitements supplémentaires pour convertir le paramètres aux contrôleurs mais leur rôle est de moins en moins pertinent dans des projets modernes. Symfony encourage à utiliser des attributs PHP pour des configurations explicites.

Une augmentation due à des **templates** enrichis, mais maîtrisée par des optimisations spécifiques.

4.4) Liste des utilisateurs

	Avant amélioration			Après amélioration		
Performances metrics	306 ms	71 ms	2.00 MB	Total execution time	Symfony initialization	Peak memory usage
	Total execution time	Symfony initialization	Peak memory usage	226 ms	22 ms	4.00 MiB

Après amélioration, le temps d'exécution a diminué de 26 % et l'initialisation de symfony à 22 ms. La mémoire a doublé, un compromis justifié par l'ajout de nouvelles fonctionnalités.

Lignes à mettre en avant dans la timeline :

	Avant amélioration	Après amélioration
Firewall Security	Symfony\Component\Security\Http\Firewall 96 ms / 2 MB	TraceableFirewallListener 2.2 ms / 4 MiB
templates		@KnpPaginator/Pagination/bootstrap_v5_pagination.html.twig 25.5 ms / 4 MiB

Firewall Security montre une meilleure gestion des règles de sécurité.

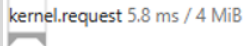
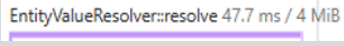
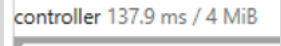
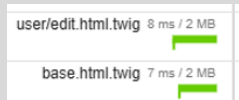
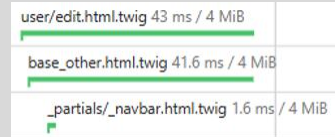
La présence des **templates** liés à la **pagination** souligne un enrichissement fonctionnel.

4.5) Modification d'un utilisateur

	Avant amélioration			Après amélioration		
Performances metrics	327 ms	91 ms	2.00 MB	Total execution time	Symfony initialization	Peak memory usage
	Total execution time	Symfony initialization	Peak memory usage	245 ms	24 ms	4.00 MiB

Le temps d'exécution a réduit de 25 %, tandis que l'initialisation de Symfony a été à 24 ms. La mémoire utilisée a encore doublé, justifiant par l'ajout de fonctionnalités et une meilleure gestion des dépendances, offrant ainsi une application plus rapide et évolutive.

Lignes à mettre en avant dans la timeline :

	Avant amélioration	Après amélioration
kernel.request		
EntityValueResolver		
controller		
templates		

Réduction significative de **kernel.request** grâce à une optimisation des des middlewares et des listeners.

Le temps lié à la résolution des arguments **EntityValueResolver ::resolve** et au **controller** reflète une augmentation de la complexité, mais cela est compensé par une meilleure modularité et une exécution plus cohérente.

Le temps passé sur le rendu des **templates** a augmenté, mais cela s'explique par des améliorations visuelles et structurelles.

4.6) Création d'un utilisateur

	Avant amélioration	Après amélioration
Performances metrics	402 ms Total execution time 35 ms Symfony initialization 4.00 MB Peak memory usage	Total execution time 119 ms Symfony initialization 11 ms Peak memory usage 4.00 MiB

Après amélioration, le temps d'exécution a été réduit à 119 ms, marquant une amélioration de 70 %, cela montre une gestion plus efficace des ressources et une application nettement plus performante.

Lignes à mettre en avant dans la timeline :

	Avant amélioration	Après amélioration
controller		

Réduction drastique concernant le **controller** montrant que les responsabilités ont été déplacées vers des services dédiés.

4.7) Conclusion des performances

L'audit des performances de l'application **ToDo & Co** avant et après optimisation montre des améliorations significatives, confirmant une meilleure rapidité, robustesse et adaptation aux standards modernes.

Optimisations Majeures :

Le temps d'exécution des pages a été considérablement réduit, avec des diminutions de plus de **50 %**, grâce à une meilleure gestion des ressources et aux optimisations de **Symfony 6.3**. La phase d'initialisation du framework est désormais plus rapide, notamment grâce à une gestion optimisée des **middlewares et événements**.

Gestion des Contrôleurs et Templates :

Malgré les gains globaux, les **temps d'exécution des contrôleurs** ont légèrement augmenté en raison de l'ajout de nouvelles fonctionnalités (résolution d'arguments, validations). De même, les **temps de rendu des templates** sont plus longs en raison d'interfaces enrichies et d'éléments dynamiques, améliorant cependant l'expérience utilisateur.

Stabilité et Gestion Mémoire :

La mémoire consommée a **doublé** (de **2 MB à 4 MB**), conséquence des nouvelles fonctionnalités comme la **pagination**. Toutefois, cette augmentation n'a pas d'impact négatif sur la réactivité.

Modernisation et Pérennité :

La migration de **Symfony 3.1 vers Symfony 6.3** garantit non seulement des performances accrues, mais aussi une meilleure sécurité et compatibilité avec les outils modernes, rendant l'application **plus maintenable et évolutive**.